Python Interview Questions

Top 100 Q&A with Code Examples

Complete Reference for Developers & Job Seekers

Table of Contents

Section 1: Python Basics & Syntax (Q1-Q10)
Section 2: Object-Oriented Programming (Q11-Q20)
Section 3: Functions, Closures & Decorators (Q21-Q30)
Section 4: Data Structures & Collections (Q31-Q40)
Section 5: Strings & Text Processing (Q41-Q45)
Section 6: File I/O, Exceptions & Modules (Q46-Q50)
Section 7: Concurrency & Async Programming (Q51-Q55)
Section 8: Memory Management & Performance (Q56-Q60)
Section 9: Testing & Debugging (Q61-Q65)
Section 10: Advanced Python Concepts (Q66-Q75)
Section 11: Popular Libraries & Frameworks (Q76-Q80)
Section 12: Design Patterns & Best Practices (Q81-Q85)
Section 13: Virtual Environments & Packaging (Q86-Q90)
Section 14: Data Serialization & Databases (Q91-Q95)
Section 15: Common Interview Coding Problems (Q96-Q100)

Section 1: Python Basics & Syntax

Q1. What is Python and what are its key features?

A: Python is a high-level, interpreted, general-purpose programming language known for its clear syntax and readability. Key features include dynamic typing, automatic memory management (garbage collection), extensive standard library, cross-platform support, object-oriented and functional programming support.

Q2. What is PEP 8 and why is it important?

A: PEP 8 is Python's official style guide that defines coding conventions — indentation (4 spaces), naming conventions, line length (79 chars), whitespace usage, and more. It promotes readable, consistent code across the Python community.

Example: PEP 8 compliant naming

```
# Functions: snake_case
def calculate_total(price, tax_rate):
    return price * (1 + tax_rate)

# Classes: PascalCase
class ShoppingCart:
    pass

# Constants: UPPER_CASE
MAX_ITEMS = 100
```

Q3. What are Python's built-in data types?

A: Python has several built-in types: int, float, complex (numeric); str (text); list, tuple, range (sequences); dict (mapping); set, frozenset (sets); bool (boolean); bytes, bytearray (binary).

Example:

```
x = 42            # int
y = 3.14          # float
name = 'Python'   # str
items = [1, 2, 3] # list
point = (0, 0)    # tuple
data = {'a': 1}   # dict
unique = {1,2,3}  # set
flag = True       # bool
```

Q4. What is the difference between a list and a tuple?

A: Lists are mutable (can be changed after creation) and use square brackets. Tuples are immutable (cannot be changed) and use parentheses. Tuples are faster, use less memory, and are hashable so they can be dict keys or set elements.

Example:

```
my_list = [1, 2, 3]
my_list[0] = 99        # OK - mutable

my_tuple = (1, 2, 3)
# my_tuple[0] = 99    # TypeError - immutable

# Tuple as dict key (valid):
coords = {(0, 0): 'origin'}
```

Q5. What is the difference between '==' and 'is'?

A: '==' checks value equality (do they have the same value?). 'is' checks identity (are they the same object in memory?). Use 'is' only for None comparisons.

Example:

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)   # True  - same value
print(a is b)   # False - different objects

x = None
print(x is None)  # True - correct way
```

Q6. What are Python's mutable and immutable types?

A: Immutable types: int, float, str, tuple, frozenset, bool, bytes. Mutable types: list, dict, set, bytearray. Immutable objects cannot be changed after creation; any 'modification' creates a new object.

Example:

```
s = 'hello'
id_before = id(s)
s += ' world'
print(id_before == id(s))  # False - new object!

lst = [1, 2]
id_before = id(lst)
lst.append(3)
print(id_before == id(lst)) # True - same object
```

Q7. What are *args and **kwargs?

A: *args allows a function to accept any number of positional arguments (collected as a tuple). **kwargs allows any number of keyword arguments (collected as a dict). They enable flexible function signatures.

Example:

```
def show(*args, **kwargs):
    print('args:', args)
    print('kwargs:', kwargs)

show(1, 2, 3, name='Alice', age=30)
# args: (1, 2, 3)
# kwargs: {'name': 'Alice', 'age': 30}
```

Q8. What is a lambda function?

A: A lambda is an anonymous, single-expression function defined inline with the lambda keyword. It's useful for short operations passed as arguments to higher-order functions.

Example:

```
# Regular function
def square(x): return x * x

# Lambda equivalent
square = lambda x: x * x

# Common use with sorted()
data = [('Bob', 25), ('Alice', 30), ('Eve', 20)]
data.sort(key=lambda x: x[1])
print(data)  # sorted by age
```

Q9. What is a list comprehension?

A: List comprehensions provide a concise way to create lists from iterables, optionally filtering with a condition. They are generally faster than equivalent for loops.

Example:

```
# Basic
squares = [x**2 for x in range(10)]

# With condition
evens = [x for x in range(20) if x % 2 == 0]

# Nested
matrix = [[i*j for j in range(3)] for i in range(3)]

# Dict comprehension
sq_dict = {x: x**2 for x in range(5)}
```

Q10. What is the difference between range() and xrange() in Python 3?

A: In Python 3, xrange() no longer exists. range() returns a range object (lazy/iterator-like), not a list. It generates numbers on demand, making it memory-efficient for large ranges. Use list(range(n)) to get an actual list.

Example:

```
r = range(1_000_000)  # No memory for 1M ints allocated
print(r[500_000])      # Direct access works
print(500_000 in r)    # O(1) membership test
print(list(range(5)))  # [0, 1, 2, 3, 4]
```

Section 2: Object-Oriented Programming

Q11. What are the four pillars of OOP in Python?

A: Encapsulation (bundling data and methods, hiding internals), Inheritance (deriving a class from another), Polymorphism (same interface, different behavior), Abstraction (exposing only necessary details).

Example: Inheritance & Polymorphism

```
class Animal:
    def speak(self): return 'Some sound'

class Dog(Animal):
    def speak(self): return 'Woof!'

class Cat(Animal):
    def speak(self): return 'Meow!'

animals = [Dog(), Cat()]
for a in animals:
    print(a.speak())  # Polymorphism
```

Q12. What is __init__ and __new__ in Python?

A: __new__ creates the instance (allocates memory); __init__ initializes it. You almost always override __init__. Override __new__ when subclassing immutable types like int or str, or for custom object creation patterns.

Example:

```
class MyClass:
    def __new__(cls, *args, **kwargs):
        print('Creating instance')
        return super().__new__(cls)

    def __init__(self, value):
        print('Initializing')
        self.value = value

obj = MyClass(42)
```

Q13. What is the difference between @classmethod, @staticmethod, and instance methods?

A: Instance methods take 'self' (the instance). Class methods take 'cls' (the class) and can access/modify class state. Static methods take neither — they're utility functions logically grouped in the class.

Example:

```
class Counter:
    count = 0

    def increment(self):       # instance method
        Counter.count += 1

    @classmethod
    def reset(cls):             # class method
        cls.count = 0

    @staticmethod
    def about():               # static method
        return 'A counter class'
```

Q14. What is multiple inheritance and the MRO (Method Resolution Order)?

A: Python supports multiple inheritance. MRO determines the order in which base classes are searched when looking up a method. Python uses the C3 linearization algorithm. Use ClassName.__mro__ to inspect it.

Example:

```
class A:
    def hello(self): return 'A'
class B(A):
    def hello(self): return 'B'
class C(A):
    def hello(self): return 'C'
class D(B, C): pass

print(D().hello())  # B (MRO: D -> B -> C -> A)
print(D.__mro__)
```

Q15. What are dunder (magic) methods in Python?

A: Dunder (double underscore) methods like __str__, __len__, __add__, __eq__ let you define how objects behave with built-in operations. They implement operator overloading and protocol support.

Example:

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):
        return Vector(self.x+other.x, self.y+other.y)
    def __repr__(self):
        return f'Vector({self.x}, {self.y})'
    def __len__(self):
        return 2

v = Vector(1,2) + Vector(3,4)
print(v)  # Vector(4, 6)
```

Q16. What is the difference between __str__ and __repr__?

A: __repr__ is for developers — should return an unambiguous string that ideally could recreate the object. __str__ is for end users — a human-readable description. repr() falls back to __repr__ if __str__ is not defined.

Example:

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __repr__(self):
        return f'Point({self.x!r}, {self.y!r})'
    def __str__(self):
        return f'({self.x}, {self.y})'

p = Point(1, 2)
print(repr(p))  # Point(1, 2)  <- developer
print(str(p))   # (1, 2)       <- user
```

Q17. What is a property decorator?

A: @property lets you define a method that is accessed like an attribute. Combined with @setter and @deleter, you get getter/setter behavior without changing the public interface.

Example:

```
class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius

    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self._celsius = (value - 32) * 5/9

t = Temperature(100)
print(t.fahrenheit)  # 212.0
```

Q18. What is __slots__ and when should you use it?

A: __slots__ restricts instance attributes to a fixed set, replacing the per-instance __dict__ with a compact structure. This reduces memory usage (up to 40-50%) and slightly speeds up attribute access. Use for classes with many instances.

Example:

```
class PointSlots:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x; self.y = y

class PointDict:
    def __init__(self, x, y):
        self.x = x; self.y = y

# PointSlots uses ~40% less memory than PointDict
```

Q19. What is the difference between shallow copy and deep copy?

A: A shallow copy creates a new object but inserts references to the objects found in the original. A deep copy creates a new object and recursively copies all nested objects. Use copy.copy() vs copy.deepcopy().

Example:

```
import copy
original = [[1, 2], [3, 4]]

shallow = copy.copy(original)
deep = copy.deepcopy(original)

original[0][0] = 99
```

```
print(shallow[0][0])  # 99  - shared reference
print(deep[0][0])     # 1   - independent copy
```

Q20. What is an abstract class in Python?

A: Abstract classes (via abc module) cannot be instantiated and may contain abstract methods that subclasses must implement. They define a common interface/contract for a group of related classes.

Example:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...

    @abstractmethod
    def perimeter(self) -> float: ...

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r**2
    def perimeter(self): return 2 * 3.14159 * self.r
```

Section 3: Functions, Closures & Decorators

Q21. What is a closure in Python?

A: A closure is a nested function that captures variables from its enclosing scope even after the outer function has returned. The inner function 'closes over' those free variables.

Example:

```
def make_multiplier(factor):
    def multiply(x):       # closes over 'factor'
        return x * factor
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(5))  # 10
print(triple(5))  # 15
```

Q22. What is a decorator and how do you create one?

A: A decorator is a callable that takes a function and returns a modified version. It uses the @syntax as syntactic sugar. functools.wraps preserves the wrapped function's metadata.

Example:

```
import functools, time

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f'{func.__name__} took {time.perf_counter()-start:.4f}s')
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(0.1)
```

Q23. What are generators and how do they work?

A: Generators are functions that use 'yield' to produce a sequence of values lazily. Each call to next() resumes execution after the last yield. They are memory-efficient for large sequences.

Example:

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

gen = fibonacci()
print([next(gen) for _ in range(8)])
# [0, 1, 1, 2, 3, 5, 8, 13]
```

Q24. What is the difference between a generator and an iterator?

A: Every generator is an iterator, but not every iterator is a generator. Iterators implement __iter__ and __next__. Generators are created with yield and automatically implement the iterator protocol. Generators are the easiest way to create iterators.

Example:

```
# Manual iterator
class CountUp:
    def __init__(self, n): self.n, self.i = n, 0
    def __iter__(self): return self
    def __next__(self):
        if self.i >= self.n: raise StopIteration
        self.i += 1; return self.i

# Generator equivalent
def count_up(n):
    for i in range(1, n+1): yield i
```

Q25. What is functools.lru_cache and when should you use it?

A: lru_cache is a decorator that memoizes function results using a Least Recently Used cache. It's ideal for pure functions with expensive computations and repeated calls with the same arguments (e.g., recursive algorithms).

Example:

```
from functools import lru_cache

@lru_cache(maxsize=128)
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)

print(fib(100))   # Fast! Results are cached
print(fib.cache_info())  # hits, misses, size
```

Q26. What is a context manager and how do you create one?

A: Context managers manage resources via the 'with' statement. They guarantee cleanup (e.g., file closure) via __enter__ and __exit__. The contextlib.contextmanager decorator offers a simpler generator-based approach.

Example:

```
from contextlib import contextmanager

@contextmanager
def managed_resource(name):
    print(f'Acquiring {name}')
```

```
    try:
        yield name
    finally:
        print(f'Releasing {name}')

with managed_resource('DB connection') as r:
    print(f'Using {r}')
```

Q27. What is the difference between map(), filter(), and reduce()?

A: map() applies a function to every item in an iterable. filter() selects items where a function returns True. reduce() (from functools) accumulates items into a single value. All return lazy iterators (except reduce which returns a value).

Example:

```
from functools import reduce

nums = [1, 2, 3, 4, 5]
squared  = list(map(lambda x: x**2, nums))      # [1,4,9,16,25]
evens    = list(filter(lambda x: x%2==0, nums)) # [2,4]
total    = reduce(lambda a, b: a+b, nums)        # 15
```

Q28. What is the difference between global and nonlocal?

A: 'global' declares that a variable inside a function refers to the module-level global. 'nonlocal' declares that a variable refers to the nearest enclosing scope (not global). Both allow modification of outer-scope variables.

Example:

```
count = 0
def increment_global():
    global count
    count += 1

def make_counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc
```

Q29. What is yield from in Python?

A: 'yield from' delegates to a sub-generator, forwarding values both in and out. It simplifies generator composition and is essential for coroutine chaining in asyncio.

Example:

```
def chain(*iterables):
    for it in iterables:
        yield from it

print(list(chain([1,2], [3,4], [5,6])))
# [1, 2, 3, 4, 5, 6]
```

Q30. What are Python's first-class functions?

A: Functions are first-class objects in Python: they can be assigned to variables, stored in data structures, passed as arguments, and returned from other functions. This enables higher-order programming patterns.

Example:

```
def apply(func, value):
    return func(value)
```

```
operations = {'double': lambda x: x*2,
              'square': lambda x: x**2}

for name, op in operations.items():
    print(f'{name}(5) = {apply(op, 5)}')
```

Section 4: Data Structures & Collections

Q31. How does Python's dict work internally?

A: Dicts are hash tables. Keys are hashed; the hash determines a slot. On collision, open addressing is used. Python 3.7+ maintains insertion order. Average O(1) for get/set/delete. Resize occurs at ~2/3 capacity.

Example:

```
d = {'a': 1, 'b': 2, 'c': 3}
print(list(d.keys()))    # ['a', 'b', 'c'] - ordered!

# Dict operations: all O(1) average
d['d'] = 4               # insert
val = d.get('e', 0)      # safe get with default
d.update({'f': 6})       # merge
```

Q32. What is the difference between a set and a frozenset?

A: set is mutable; frozenset is immutable. frozensets are hashable and can be used as dict keys or set elements. Both use hash tables and provide O(1) average membership testing.

Example:

```
s = {1, 2, 3}
s.add(4)              # OK

fs = frozenset([1, 2, 3])
# fs.add(4)          # AttributeError

# frozenset as dict key:
graph = {frozenset({1,2}): 'edge'}
print(graph)
```

Q33. What is collections.defaultdict?

A: defaultdict is a dict subclass that calls a factory function to supply missing values. It eliminates KeyError and simplifies code that builds dicts of lists, counts, etc.

Example:

```
from collections import defaultdict

# Group words by first letter
by_letter = defaultdict(list)
for word in ['apple','banana','avocado','cherry']:
    by_letter[word[0]].append(word)

print(dict(by_letter))
# {'a': ['apple','avocado'], 'b':['banana'], 'c':['cherry']}
```

Q34. What is collections.Counter?

A: Counter is a dict subclass for counting hashable objects. It supports arithmetic operations (add, subtract, intersection, union) on counts and methods like most_common().

Example:

```
from collections import Counter
```

```
text = 'abracadabra'
c = Counter(text)
print(c.most_common(3))  # [('a', 5), ('b', 2), ('r', 2)]

# Arithmetic
c2 = Counter('aab')
print(c + c2)   # combines counts
print(c - c2)   # subtracts
```

Q35. What is collections.deque and when should you use it?

A: deque (double-ended queue) is a thread-safe list-like container with O(1) appends and pops from both ends (vs. O(n) for list.insert(0,...)). Use for queues, sliding windows, and BFS algorithms.

Example:

```
from collections import deque

dq = deque([1, 2, 3], maxlen=5)
dq.appendleft(0)   # O(1) - deque: [0,1,2,3]
dq.popleft()       # O(1) - deque: [1,2,3]

# Sliding window of last 3
window = deque(maxlen=3)
for x in range(10):
    window.append(x)
print(window)  # deque([7, 8, 9])
```

Q36. What is a namedtuple and when should you use it?

A: namedtuple creates tuple subclasses with named fields. They are immutable, memory-efficient (like tuples), and more readable than plain tuples. Use for simple record types without methods.

Example:

```
from collections import namedtuple

Point = namedtuple('Point', ['x', 'y', 'z'])
p = Point(1, 2, 3)

print(p.x, p.y)       # Attribute access
print(p[0], p[1])     # Index access (tuple)
x, y, z = p           # Unpacking
print(p._asdict())    # OrderedDict
```

Q37. What is the heapq module?

A: heapq provides a min-heap implementation on top of Python lists. Common operations: heappush, heappop (O(log n)), heapify (O(n)), nlargest, nsmallest. Use for priority queues and sorting algorithms.

Example:

```
import heapq

heap = []
heapq.heappush(heap, 5)
heapq.heappush(heap, 1)
heapq.heappush(heap, 3)
print(heapq.heappop(heap))  # 1 (minimum)

# Top 3 largest in O(n log k)
data = [3,1,4,1,5,9,2,6]
print(heapq.nlargest(3, data))  # [9, 6, 5]
```

Q38. What is the difference between list.sort() and sorted()?

A: list.sort() sorts in-place (returns None). sorted() returns a new sorted list and works on any iterable. Both accept key and reverse parameters. list.sort() is slightly faster as it avoids creating a new list.

Example:

```
data = [3, 1, 4, 1, 5, 9]

s = sorted(data)           # new list
print(data)                # [3, 1, 4, 1, 5, 9] unchanged

data.sort()                # in-place
print(data)                # [1, 1, 3, 4, 5, 9]

# Sort objects by attribute
people = [('Alice',30),('Bob',25)]
people.sort(key=lambda p: p[1])
```

Q39. What is a dict comprehension and set comprehension?

A: Like list comprehensions, dict and set comprehensions offer concise syntax for creating these containers from iterables.

Example:

```
# Dict comprehension
sq = {x: x**2 for x in range(5)}
# {0:0, 1:1, 2:4, 3:9, 4:16}

# Inverted dict
original = {'a': 1, 'b': 2}
inverted = {v: k for k, v in original.items()}

# Set comprehension
unique_lens = {len(w) for w in ['hi','hello','hey']}
```

Q40. How does Python's list work internally?

A: Python lists are dynamic arrays. They pre-allocate extra capacity to amortize appending. Indexing is O(1). Appending is amortized O(1). Inserting/deleting at the front is O(n) because elements must shift.

Complexity summary:

```
# list complexities:
# append(x)      O(1) amortized
# insert(0, x)   O(n) - shifts all elements
# pop()          O(1)
# pop(0)         O(n)
# x in lst       O(n) - linear search
# lst[i]         O(1)

# Prefer deque for front operations!
```

Section 5: Strings & Text Processing

Q41. What are Python's string formatting options?

A: Python has four string formatting styles: % operator (old style), str.format() (new style), f-strings (Python 3.6+, fastest and most readable), and Template strings (for user-supplied templates).

Example:

```
name, score = 'Alice', 98.5

# % style (legacy)
print('Name: %s, Score: %.1f' % (name, score))

# .format()
```

```
print('Name: {}, Score: {:.1f}'.format(name, score))

# f-string (preferred)
print(f'Name: {name}, Score: {score:.1f}')

# f-string with expression
print(f'{2**10 = }')  # 2**10 = 1024
```

Q42. What are the most useful string methods?

A: Key methods: strip/lstrip/rstrip (whitespace), split/join, upper/lower/title/capitalize, replace, find/index, startswith/endswith, encode/decode, isalpha/isdigit/isalnum.

Example:

```
s = '  Hello, World!  '
print(s.strip())             # 'Hello, World!'
print(s.lower().strip())     # 'hello, world!'
print('x'.join(['a','b','c'])) # 'axbxc'
print('a,b,c'.split(','))    # ['a','b','c']
print('hello'.replace('l','r')) # 'herro'
print('abc'.startswith('ab'))  # True
```

Q43. What is string interning in Python?

A: Python automatically interns (caches) short strings and identifiers. Two variables assigned the same interned string share the same object. sys.intern() can manually intern strings. This explains why 'is' may return True for short strings but is still unreliable for equality checking.

Example:

```
import sys
a = sys.intern('hello world')
b = sys.intern('hello world')
print(a is b)   # True - same object

# Without intern:
x = 'hello world'
y = 'hello world'
print(x is y)   # May be True or False (CPython detail)
```

Q44. What is the re module and how do you use it?

A: The re module provides regular expression support. Compile patterns with re.compile() for reuse. Common functions: match (start), search (anywhere), findall, finditer, sub (replace), split.

Example:

```
import re

text = 'Prices: $10.99, $5.50, $120.00'
pattern = re.compile(r'\$([\d.]+)')

prices = pattern.findall(text)
print(prices)  # ['10.99', '5.50', '120.00']

# Replace
clean = re.sub(r'\s+', ' ', 'too   many  spaces')
print(clean)  # 'too many spaces'
```

Q45. What is the difference between str.encode() and bytes.decode()?

A: str.encode(encoding) converts a string to bytes using the specified encoding (default UTF-8). bytes.decode(encoding) converts bytes back to a string. This is essential when working with files, networks, or APIs.

Example:

```
text = 'Hello, 世界'
encoded = text.encode('utf-8')
print(encoded)           # b'Hello, \xe4\xb8\x96\xe7\x95\x8c'
print(type(encoded))     # <class 'bytes'>

decoded = encoded.decode('utf-8')
print(decoded == text)   # True
```

Section 6: File I/O, Exceptions & Modules

Q46. How do you read and write files in Python?

A: Use the built-in open() function with a context manager to ensure files are properly closed. Modes: 'r' (read), 'w' (write/overwrite), 'a' (append), 'b' (binary), 'x' (exclusive create).

Example:

```
# Writing
with open('data.txt', 'w', encoding='utf-8') as f:
    f.write('Hello\n')
    f.writelines(['line2\n','line3\n'])

# Reading all at once
with open('data.txt', 'r') as f:
    content = f.read()

# Reading line by line (memory-efficient)
with open('data.txt') as f:
    for line in f:
        print(line.strip())
```

Q47. What is exception handling in Python?

A: Use try/except/else/finally. 'except' catches exceptions. 'else' runs if no exception occurred. 'finally' always runs (cleanup). Raise custom exceptions by subclassing Exception.

Example:

```
class InsufficientFundsError(ValueError):
    pass

def withdraw(balance, amount):
    try:
        if amount > balance:
            raise InsufficientFundsError(f'Need {amount}, have {balance}')
        return balance - amount
    except InsufficientFundsError as e:
        print(f'Error: {e}')
    else:
        print('Withdrawal successful')
    finally:
        print('Transaction complete')
```

Q48. What is the difference between 'raise' and 'raise from'?

A: 'raise' re-raises an exception or raises a new one. 'raise X from Y' chains exceptions, setting __cause__ to Y (explicit chaining). 'raise X from None' suppresses the context, hiding the original exception.

Example:

```
try:
    int('abc')
except ValueError as e:
    raise RuntimeError('Failed to parse config') from e
```

```
    # Shows both ValueError and RuntimeError in traceback

# Suppress chaining:
try:
    int('abc')
except ValueError:
    raise RuntimeError('Bad config') from None
```

Q49. What are Python modules and packages?

A: A module is a .py file. A package is a directory with an __init__.py file. Packages organize modules hierarchically. Use import, from...import, or importlib for dynamic importing.

Example:

```
# mypackage/__init__.py
# mypackage/utils.py
# mypackage/models/user.py

from mypackage.utils import helper
from mypackage.models.user import User
import mypackage.utils as utils

# Relative import (inside package)
from ..utils import helper   # go up one level
```

Q50. What is the if __name__ == '__main__' idiom?

A: When Python runs a file, it sets __name__ to '__main__' for the entry point, but to the module name if imported. This guard ensures code only runs when the script is executed directly, not when imported.

Example:

```
# mymodule.py
def greet(name):
    return f'Hello, {name}!'

if __name__ == '__main__':
    # Only runs when executed directly:
    # python mymodule.py
    print(greet('World'))

# When imported, greet() is available but
# the print() never executes.
```

Section 7: Concurrency & Async Programming

Q51. What is the GIL (Global Interpreter Lock)?

A: The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time. It simplifies memory management but limits true CPU parallelism for multi-threaded Python code. CPU-bound tasks should use multiprocessing; I/O-bound tasks work fine with threading.

Impact summary:

```
# GIL impact:
# - I/O-bound tasks (network, disk): threading works fine
#   (GIL released during I/O waits)
# - CPU-bound tasks (computation): use multiprocessing
#   (each process has its own GIL)
# - asyncio: cooperative multitasking, single thread,
#   no GIL issue
```

Q52. What is asyncio and how does it work?

A: asyncio is Python's async framework using cooperative multitasking. The event loop drives coroutines (async def functions). 'await' suspends a coroutine and hands control back to the loop. Great for I/O-bound concurrent code.

Example:

```
import asyncio

async def fetch(url):
    await asyncio.sleep(0.1)  # simulates I/O
    return f'data from {url}'

async def main():
    # Run concurrently
    results = await asyncio.gather(
        fetch('url1'), fetch('url2'), fetch('url3')
    )
    print(results)

asyncio.run(main())
```

Q53. What is the difference between threading and multiprocessing?

A: threading: multiple threads share memory within one process. Limited by GIL for CPU tasks. multiprocessing: separate processes with separate memory. True parallelism for CPU-bound tasks. Communication via Queue/Pipe. threading is lightweight; multiprocessing has higher overhead.

Example:

```
from multiprocessing import Pool

def cpu_work(n):
    return sum(i*i for i in range(n))

# Uses multiple CPU cores
with Pool() as pool:
    results = pool.map(cpu_work, [10**6]*4)
print(results)
```

Q54. What is concurrent.futures?

A: concurrent.futures provides a high-level interface for asynchronous execution via ThreadPoolExecutor (threads) and ProcessPoolExecutor (processes). Both implement the same Executor API with submit(), map(), and as_completed().

Example:

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import requests

urls = ['https://example.com'] * 5

def fetch(url):
    return len(url)  # simulate

with ThreadPoolExecutor(max_workers=5) as ex:
    futures = {ex.submit(fetch, u): u for u in urls}
    for f in as_completed(futures):
        print(f.result())
```

Q55. What is a thread-safe queue?

A: queue.Queue is a thread-safe FIFO queue backed by a deque and a lock. It's the recommended way to pass data between threads. Put blocks when full; get blocks when empty (with optional timeout).

Example:

```
import threading, queue

q = queue.Queue(maxsize=10)

def producer():
    for i in range(5):
        q.put(i)
        print(f'Produced {i}')

def consumer():
    while True:
        item = q.get()
        print(f'Consumed {item}')
        q.task_done()

threading.Thread(target=producer).start()
```

Section 8: Memory Management & Performance

Q56. How does Python's garbage collection work?

A: CPython primarily uses reference counting: objects are freed when their count drops to 0. A cyclic garbage collector handles reference cycles. gc module controls it. PyPy and other implementations may differ.

Example:

```
import gc, sys

x = [1, 2, 3]
print(sys.getrefcount(x))  # 2 (x + argument)

# Cyclic reference example:
a = []
b = [a]
a.append(b)     # cycle: a -> b -> a
del a, b        # refcount doesn't hit 0
gc.collect()    # cyclic GC cleans it up
print('Cycles collected')
```

Q57. How do you profile Python code?

A: Use cProfile for function-level profiling, timeit for micro-benchmarks, line_profiler for line-level profiling, memory_profiler for memory, and tracemalloc for memory tracing.

Example:

```
import cProfile, pstats, io

profiler = cProfile.Profile()
profiler.enable()

# Code to profile
total = sum(i**2 for i in range(100_000))

profiler.disable()
s = io.StringIO()
ps = pstats.Stats(profiler, stream=s).sort_stats('cumulative')
ps.print_stats(5)  # top 5 functions
print(s.getvalue())
```

Q58. What are common ways to optimize Python performance?

A: Key optimizations: use built-in functions (written in C), prefer list comprehensions over loops, use local variables in hot loops, choose the right data structure, avoid unnecessary function calls, use numpy for numerical work, use Cython or C extensions for critical paths.

Example:

```
# Slow: repeated global lookup
import math
def slow():
    return [math.sqrt(x) for x in range(10000)]

# Faster: local reference
def fast():
    sqrt = math.sqrt  # single lookup
    return [sqrt(x) for x in range(10000)]
```

Q59. What is the difference between is and == for None checking?

A: Always use 'is None' or 'is not None' to check for None. None is a singleton, so identity check is correct and faster. Using '== None' works but is slower and can be confused by objects overriding __eq__.

Example:

```
value = None

# Correct:
if value is None:
    print('No value')

# Also correct:
if value is not None:
    print('Has value')

# Avoid:
if value == None:   # works but not idiomatic
    pass
```

Q60. What is sys.getsizeof and how do you measure memory?

A: sys.getsizeof returns the memory size of an object in bytes (not including referenced objects). For deep measurement, use third-party tools like pympler or tracemalloc.

Example:

```
import sys
print(sys.getsizeof([]))         # 56 bytes (empty list)
print(sys.getsizeof([1,2,3]))    # 88 bytes
print(sys.getsizeof({}))         # 64 bytes
print(sys.getsizeof('hello'))    # 54 bytes

# Note: does NOT count objects inside containers!
# Use pympler.asizeof for total deep size
```

Section 9: Testing & Debugging

Q61. How do you write unit tests in Python?

A: Python has the built-in unittest module (xUnit style). pytest is the most popular testing framework, with simpler syntax, better fixtures, and rich plugins. Both discover test files automatically.

Example (pytest):

```
# test_calculator.py
def add(a, b): return a + b
def divide(a, b):
    if b == 0: raise ValueError('Cannot divide by zero')
```

```
    return a / b

def test_add():
    assert add(2, 3) == 5

def test_divide_by_zero():
    import pytest
    with pytest.raises(ValueError):
        divide(1, 0)
```

Q62. What is mocking in Python testing?

A: unittest.mock provides Mock and MagicMock objects to replace parts of the system under test. @patch decorator replaces real objects temporarily. Mocks record calls and can be configured to return specific values.

Example:

```
from unittest.mock import patch, MagicMock

def get_user(user_id):
    return db.fetch(user_id)  # real DB call

def test_get_user():
    with patch('mymodule.db') as mock_db:
        mock_db.fetch.return_value = {'id': 1, 'name': 'Alice'}
        result = get_user(1)
        assert result['name'] == 'Alice'
        mock_db.fetch.assert_called_once_with(1)
```

Q63. What is a pytest fixture?

A: Fixtures are functions decorated with @pytest.fixture that provide test setup/teardown. They are dependency-injected into test functions by name. Fixtures support scoping (function, class, module, session) and yielding for teardown.

Example:

```
import pytest

@pytest.fixture
def sample_data():
    data = {'users': ['Alice', 'Bob']}
    yield data          # setup complete
    data.clear()        # teardown (runs after test)

def test_users(sample_data):
    assert 'Alice' in sample_data['users']
```

Q64. What is the pdb debugger?

A: pdb is Python's built-in debugger. Set breakpoints with breakpoint() (Python 3.7+) or import pdb; pdb.set_trace(). Commands: n (next), s (step into), c (continue), l (list code), p (print), b (breakpoint), q (quit).

Example:

```
def buggy_function(data):
    result = []
    for item in data:
        breakpoint()    # drops into pdb here
        processed = item * 2
        result.append(processed)
    return result

# Run with: python myscript.py
# Or: python -m pdb myscript.py
```

Q65. What is the logging module and how should you use it?

A: The logging module is the standard way to output diagnostic information. Use it instead of print() in production code. Levels: DEBUG, INFO, WARNING, ERROR, CRITICAL. Configure with basicConfig or a logging.config dict.

Example:

```
import logging

logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s %(name)s %(levelname)s %(message)s'
)
logger = logging.getLogger(__name__)

logger.debug('Detailed info')
logger.info('Normal operation')
logger.warning('Something unexpected')
logger.error('A problem occurred')
```

Section 10: Advanced Python Concepts

Q66. What are metaclasses in Python?

A: A metaclass is the class of a class. It controls class creation. The default metaclass is 'type'. Custom metaclasses can modify class attributes, enforce interfaces, add methods automatically, or implement singletons.

Example:

```
class SingletonMeta(type):
    _instances = {}
    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]

class Config(metaclass=SingletonMeta):
    pass

a, b = Config(), Config()
print(a is b)   # True - same instance
```

Q67. What is a descriptor in Python?

A: A descriptor is an object that defines __get__, __set__, or __delete__. When a descriptor is set as a class attribute, Python calls these methods instead of directly accessing the attribute. Properties are implemented as descriptors.

Example:

```
class Positive:
    def __set_name__(self, owner, name):
        self.name = name
    def __get__(self, obj, type=None):
        return obj.__dict__.get(self.name)
    def __set__(self, obj, value):
        if value <= 0:
            raise ValueError(f'{self.name} must be positive')
        obj.__dict__[self.name] = value

class Product:
    price = Positive()
```

Q68. What is dataclasses module?

A: dataclasses (Python 3.7+) auto-generates boilerplate like __init__, __repr__, __eq__ from class annotations. Use @dataclass. Supports default values, field() customization, frozen (immutable), post_init hooks.

Example:

```
from dataclasses import dataclass, field

@dataclass(order=True, frozen=True)
class Point:
    x: float
    y: float
    z: float = 0.0
    tags: list = field(default_factory=list)

p1 = Point(1.0, 2.0)
p2 = Point(3.0, 4.0)
print(p1 < p2)    # True (order=True)
```

Q69. What is the typing module and type hints?

A: Type hints (PEP 484) allow annotating variable and function types. The typing module provides List, Dict, Optional, Union, Callable, etc. Type hints are not enforced at runtime but enable static analysis with mypy.

Example:

```
from typing import Optional, Union, List

def process(
    items: List[int],
    multiplier: Union[int, float] = 1.0,
    label: Optional[str] = None
) -> List[float]:
    prefix = f'{label}: ' if label else ''
    return [x * multiplier for x in items]

# Use mypy for static checking: mypy myfile.py
```

Q70. What is Protocol in Python's typing system?

A: Protocol (PEP 544, Python 3.8+) enables structural subtyping ('duck typing' with type safety). A class satisfies a Protocol if it has the required methods/attributes — no explicit inheritance needed.

Example:

```
from typing import Protocol

class Drawable(Protocol):
    def draw(self) -> None: ...

class Circle:
    def draw(self): print('O')   # no inheritance needed

class Square:
    def draw(self): print('[]')

def render(shape: Drawable):
    shape.draw()   # works for any class with draw()
```

Q71. What is __init_subclass__ and when is it useful?

A: __init_subclass__ is called on the base class whenever a new subclass is created. It lets you register subclasses, validate them, or modify them automatically without a metaclass.

Example:

```
class Plugin:
    registry = {}

    def __init_subclass__(cls, name, **kwargs):
        super().__init_subclass__(**kwargs)
        Plugin.registry[name] = cls

class AudioPlugin(Plugin, name='audio'): pass
class VideoPlugin(Plugin, name='video'): pass

print(Plugin.registry)
# {'audio': <class AudioPlugin>, 'video': ...}
```

Q72. What is the difference between @classmethod and __init_subclass__?

A: @classmethod is inherited and called explicitly. __init_subclass__ is called automatically when a subclass is defined. __init_subclass__ is better for auto-registration, framework hooks, and class validation at definition time.

Q73. What are Python's comparison operators and the functools.total_ordering decorator?

A: Normally you'd implement all 6 comparison operators (__lt__, __le__, __gt__, __ge__, __eq__, __ne__). @total_ordering fills in the missing ones if you define __eq__ and one of __lt__/__le__/__gt__/__ge__.

Example:

```
from functools import total_ordering

@total_ordering
class Version:
    def __init__(self, major, minor):
        self.major, self.minor = major, minor
    def __eq__(self, other):
        return (self.major, self.minor) == (other.major, other.minor)
    def __lt__(self, other):
        return (self.major, self.minor) < (other.major, other.minor)

v1, v2 = Version(1,0), Version(2,0)
print(v1 < v2, v1 <= v2, v1 > v2)
```

Q74. What is __call__ and callable objects?

A: __call__ makes an instance callable like a function. Useful for stateful callables, function objects that maintain state, and decorators implemented as classes.

Example:

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor
    def __call__(self, x):
        return x * self.factor

double = Multiplier(2)
triple = Multiplier(3)
print(double(5))   # 10
print(triple(5))   # 15
print(callable(double))  # True
```

Q75. What is the walrus operator (:=)?

A: The walrus operator (PEP 572, Python 3.8+) is the assignment expression operator. It assigns a value to a variable as part of an expression. Useful in while loops and comprehensions to avoid redundant computation.

Example:

```
# Without walrus:
data = expensive_call()
if data:
    process(data)

# With walrus:
if data := expensive_call():
    process(data)

# In while loop:
while chunk := f.read(8192):
    process(chunk)

# In comprehension:
results = [y for x in data if (y := transform(x)) > 0]
```

Section 11: Popular Libraries & Frameworks

Q76. What is the difference between requests and httpx?

A: requests is the classic synchronous HTTP library. httpx is a modern alternative that supports both sync and async, HTTP/2, connection pooling, and has a similar API. Use httpx for async applications or when you need HTTP/2.

Example (httpx async):

```
import asyncio, httpx

async def fetch_many(urls):
    async with httpx.AsyncClient() as client:
        tasks = [client.get(url) for url in urls]
        responses = await asyncio.gather(*tasks)
    return [r.status_code for r in responses]

urls = ['https://httpbin.org/get'] * 3
print(asyncio.run(fetch_many(urls)))
```

Q77. What is SQLAlchemy and what are its two APIs?

A: SQLAlchemy is Python's most popular ORM. It has two APIs: Core (SQL expression language, close to SQL) and ORM (maps Python classes to tables). The modern 2.0 API unifies both with a consistent session model.

Example (ORM):

```
from sqlalchemy.orm import DeclarativeBase, Session
from sqlalchemy import create_engine, String, select
from sqlalchemy.orm import Mapped, mapped_column

class Base(DeclarativeBase): pass
class User(Base):
    __tablename__ = 'user'
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50))

engine = create_engine('sqlite://')
```

Q78. What is Pydantic and how is it used?

A: Pydantic is a data validation library using Python type annotations. It validates data at runtime, converts types automatically, and generates JSON schemas. It's the backbone of FastAPI and is widely used for configuration and data parsing.

Example:

```
from pydantic import BaseModel, validator, EmailStr

class User(BaseModel):
    id: int
    name: str
    email: str
    age: int

    @validator('age')
    def age_must_be_positive(cls, v):
        if v <= 0: raise ValueError('Age must be positive')
        return v

user = User(id='1', name='Alice', email='a@b.com', age=30)
```

Q79. What is the difference between Flask and FastAPI?

A: Flask is a micro-framework, WSGI-based, synchronous by default. FastAPI is ASGI-based, async-first, with automatic OpenAPI docs, request validation via Pydantic, and dependency injection. FastAPI is significantly faster for async I/O.

FastAPI example:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

@app.post('/items/', response_model=Item)
async def create_item(item: Item):
    return item

# Auto docs at /docs and /redoc!
```

Q80. What is numpy and why is it fast?

A: NumPy provides multi-dimensional arrays and mathematical operations. It's fast because arrays are contiguous blocks of homogeneous typed memory, and operations call optimized C/Fortran routines. Vectorized operations avoid Python loops.

Example:

```
import numpy as np

# 100x faster than a Python list loop
a = np.arange(1_000_000)
b = np.arange(1_000_000)

result = a + b          # vectorized, no Python loop
result = np.sqrt(a)     # universal function (ufunc)
result = a @ b          # dot product (optimized BLAS)
```

Section 12: Design Patterns & Best Practices

Q81. What is the Singleton pattern in Python?

A: Singleton ensures only one instance of a class exists. Implement via __new__, a module-level variable, metaclass, or class decorator. In Python, modules themselves are singletons.

Example (thread-safe):

```
import threading

class Singleton:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
        return cls._instance

a, b = Singleton(), Singleton()
print(a is b)  # True
```

Q82. What is the Factory pattern?

A: Factory creates objects without specifying exact classes. It decouples creation from usage and makes the code easier to extend.

Example:

```
class Animal:
    def speak(self): ...

class Dog(Animal):
    def speak(self): return 'Woof'

class Cat(Animal):
    def speak(self): return 'Meow'

def animal_factory(kind: str) -> Animal:
    animals = {'dog': Dog, 'cat': Cat}
    cls = animals.get(kind)
    if not cls: raise ValueError(f'Unknown: {kind}')
    return cls()
```

Q83. What is the Observer pattern?

A: Observer (event system) allows objects (observers) to be notified when another object (subject) changes state. Python's property setters, Django signals, and asyncio events follow this pattern.

Example:

```
class EventEmitter:
    def __init__(self):
        self._handlers = {}
    def on(self, event, handler):
        self._handlers.setdefault(event, []).append(handler)
    def emit(self, event, *args):
        for h in self._handlers.get(event, []):
            h(*args)

emitter = EventEmitter()
emitter.on('data', lambda x: print(f'Got: {x}'))
emitter.emit('data', 42)
```

Q84. What is dependency injection in Python?

A: DI passes dependencies into an object rather than creating them inside. This improves testability and flexibility. Python doesn't need a DI framework — pass dependencies via constructors or use libraries like injector or dependency_injector.

Example:

```
class EmailService:
    def send(self, msg): print(f'Email: {msg}')

class SMSService:
    def send(self, msg): print(f'SMS: {msg}')

class Notifier:
    def __init__(self, service):   # injection!
        self.service = service
    def notify(self, msg):
        self.service.send(msg)

Notifier(EmailService()).notify('Hello')
```

Q85. What is the SOLID principle in Python context?

A: SOLID: Single Responsibility (one reason to change), Open/Closed (open for extension, closed for modification), Liskov Substitution (subclass replaces superclass safely), Interface Segregation (small interfaces), Dependency Inversion (depend on abstractions).

Open/Closed example:

```
from abc import ABC, abstractmethod

class Discount(ABC):
    @abstractmethod
    def apply(self, price): ...

class NoDiscount(Discount):
    def apply(self, price): return price

class PercentDiscount(Discount):
    def __init__(self, pct): self.pct = pct
    def apply(self, price): return price * (1 - self.pct/100)
```

Section 13: Virtual Environments & Packaging

Q86. What is a virtual environment and how do you create one?

A: A virtual environment is an isolated Python environment with its own packages. Use python -m venv, conda, or poetry. This prevents package version conflicts between projects.

Commands:

```
# Create
python -m venv .venv

# Activate (Unix/Mac)
source .venv/bin/activate

# Activate (Windows)
.venv\Scripts\activate

# Install packages
pip install requests pandas

# Freeze dependencies
pip freeze > requirements.txt
```

Q87. What is the difference between pip and poetry?

A: pip is the standard package installer. poetry is a modern dependency management tool that handles virtual environments, dependency resolution, versioning, and publishing. poetry.lock ensures reproducible installs.

Poetry commands:

```
# Init new project
poetry new myproject

# Add dependency
poetry add requests

# Add dev dependency
poetry add pytest --dev

# Install from lock file
poetry install

# Run in env
poetry run python main.py
```

Q88. What is __all__ in a Python module?

A: __all__ is a list of public names exported by a module when someone does 'from module import *'. It also serves as documentation of the module's public API. Names starting with underscore are always private.

Example:

```
# mymodule.py
__all__ = ['PublicClass', 'public_function']

class PublicClass: pass          # exported
def public_function(): pass      # exported
def _helper(): pass              # NOT exported (private)

# from mymodule import *
# -> imports only PublicClass and public_function
```

Q89. What is setup.py vs pyproject.toml?

A: setup.py was the classic way to define Python package metadata. pyproject.toml (PEP 517/518) is the modern standard, supporting multiple build backends (setuptools, flit, poetry, hatchling). Always prefer pyproject.toml for new projects.

pyproject.toml example:

```
[build-system]
requires = ['setuptools>=61']
build-backend = 'setuptools.backends.legacy:build'

[project]
name = 'mypackage'
version = '1.0.0'
dependencies = ['requests>=2.28']

[project.optional-dependencies]
dev = ['pytest', 'mypy']
```

Q90. What is the difference between requirements.txt and pyproject.toml dependencies?

A: requirements.txt pins exact versions (used for reproducible deployments, especially with pip freeze). pyproject.toml specifies version ranges (for library publishing, allowing users flexibility). Use both: pyproject.toml for the library spec, requirements.txt for deployment.

Section 14: Data Serialization & Databases

Q91. How do you work with JSON in Python?

A: The json module serializes Python objects to JSON strings/files (json.dumps/json.dump) and deserializes back (json.loads/json.load). Custom objects need a default encoder or JSONEncoder subclass.

Example:

```
import json
from datetime import datetime

class DateEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, datetime):
            return obj.isoformat()
        return super().default(obj)

data = {'name': 'Alice', 'created': datetime.now()}
s = json.dumps(data, cls=DateEncoder, indent=2)
back = json.loads(s)
```

Q92. How do you use sqlite3 in Python?

A: The built-in sqlite3 module provides a lightweight disk-based database. Use context managers for connections, parameterized queries to prevent SQL injection, and cursor objects to execute SQL.

Example:

```
import sqlite3

with sqlite3.connect('app.db') as conn:
    conn.execute('''CREATE TABLE IF NOT EXISTS users
                    (id INTEGER PRIMARY KEY, name TEXT)''')

    # Parameterized query (prevents SQL injection)
    conn.execute('INSERT INTO users (name) VALUES (?)', ('Alice',))

    cursor = conn.execute('SELECT * FROM users')
    for row in cursor:
        print(row)
```

Q93. What is the csv module?

A: The csv module reads and writes CSV files, handling quoting, escaping, and different delimiters. DictReader/DictWriter use column headers as dict keys for more convenient access.

Example:

```
import csv

# Write
with open('data.csv','w',newline='') as f:
    writer = csv.DictWriter(f, fieldnames=['name','age'])
    writer.writeheader()
    writer.writerows([{'name':'Alice','age':30}])

# Read
with open('data.csv') as f:
    for row in csv.DictReader(f):
        print(row['name'], row['age'])
```

Q94. What is pickle and when should you avoid it?

A: pickle serializes arbitrary Python objects to bytes and back. It's fast and preserves Python types. Never unpickle data from untrusted sources — it can execute arbitrary code. For data exchange, prefer JSON or msgpack.

Example:

```
import pickle
```

```
data = {'model': [1,2,3], 'params': {'lr': 0.01}}

# Serialize
with open('model.pkl', 'wb') as f:
    pickle.dump(data, f)

# Deserialize
with open('model.pkl', 'rb') as f:
    loaded = pickle.load(f)

# WARNING: Never unpickle untrusted data!
```

Q95. What is pathlib and how does it improve file handling?

A: pathlib.Path provides an object-oriented file system path interface. It's more readable and portable than os.path string manipulation, supports operator / for joining paths, and integrates with the open() function.

Example:

```
from pathlib import Path

base = Path('/home/user')
config = base / 'app' / 'config.json'  # path joining

config.parent.mkdir(parents=True, exist_ok=True)
config.write_text('{"debug": true}')

print(config.exists())      # True
print(config.suffix)        # .json
print(config.stem)          # config
for f in base.glob('**/*.py'):  # recursive glob
    print(f)
```

Section 15: Common Interview Coding Problems

Q96. How do you reverse a string in Python?

A: The Pythonic way is slicing with [::-1]. str.join(reversed()) also works. There is no built-in str.reverse() because strings are immutable.

Example:

```
s = 'Hello, World!'
print(s[::-1])                    # '!dlroW ,olleH'
print(''.join(reversed(s)))        # same

# Palindrome check
def is_palindrome(s):
    s = s.lower().replace(' ', '')
    return s == s[::-1]

print(is_palindrome('A man a plan a canal Panama'))  # True
```

Q97. How do you find duplicates in a list?

A: Multiple approaches: convert to set (O(n) time/space), use Counter, or find elements seen more than once. Counter is most flexible.

Example:

```
from collections import Counter

nums = [1, 2, 3, 2, 4, 3, 5]

# With Counter
```

```
counts = Counter(nums)
dupes = [x for x, c in counts.items() if c > 1]
print(dupes)   # [2, 3]

# With set
seen, dupes2 = set(), set()
for n in nums:
    dupes2.add(n) if n in seen else seen.add(n)
```

Q98. How do you flatten a nested list?

A: For one level deep, use itertools.chain.from_iterable. For arbitrary depth, use recursion or a generator with yield from.

Example:

```
import itertools

# One level
nested = [[1,2],[3,4],[5,6]]
flat = list(itertools.chain.from_iterable(nested))
print(flat)   # [1, 2, 3, 4, 5, 6]

# Arbitrary depth
def flatten(lst):
    for item in lst:
        if isinstance(item, list):
            yield from flatten(item)
        else:
            yield item
```

Q99. How do you implement a stack and queue in Python?

A: Stack (LIFO): use list with append/pop. Queue (FIFO): use collections.deque with append/popleft (O(1)), or queue.Queue for thread safety.

Example:

```
from collections import deque

# Stack (LIFO)
stack = []
stack.append(1); stack.append(2); stack.append(3)
print(stack.pop())    # 3 (LIFO)

# Queue (FIFO)
queue = deque()
queue.append(1); queue.append(2); queue.append(3)
print(queue.popleft())  # 1 (FIFO)
```

Q100. What is the best way to merge two dictionaries in Python?

A: Python 3.9+ supports the | operator. Python 3.5+ supports {**d1, **d2} unpacking. dict.update() modifies in-place. For nested merging, a recursive deep merge function is needed.

Example:

```
a = {'x': 1, 'y': 2}
b = {'y': 3, 'z': 4}   # 'y' conflicts

# Python 3.9+ (preferred)
merged = a | b
print(merged)  # {'x':1, 'y':3, 'z':4}  - b wins

# Python 3.5+
merged = {**a, **b}    # same result
```

```
# In-place update
a.update(b)            # modifies a
```